2021 11th International Conference on Cloud Computing, Data Science & Engineering (Confluence) | 978-1-6654-1451-7/20/$31.00 ©2021 IEEE | DOI: 10.1109/Confluence51648.2021.9377083

A Study on Machine Learning Applied to Software Bug Priority Prediction

Ruchika Malhotra
Department of Computer Science & Engineering,
Delhi Technological University
Delhi, India.
ruchikamalhotra@dtu.ac.in

Ajay Dabas
Department of Computer Science & Engineering,
Delhi Technological University
Delhi, India.
se.dabasajay@gmail.com

Hariharasudhan A S
Department of Computer Science & Engineering,
Delhi Technological University
Delhi, India.
harimailbox99@gmail.com

Manish Pant
Department of Computer Science & Engineering,
Delhi Technological University
Delhi, India.
manishpant99@gmail.com

***Abstract*—Bugs are among the top problems faced by software developers. As the size and complexity of software projects increase so does the number of bugs and their complexity. Bug priority prediction helps software developers focus their efforts on the most critical bugs that affect the core functionality of a software. By automating the process of priority prediction, it is possible to reduce the time spent analyzing new bug reports. In this paper, we extract bug reports from the bug tracking software of six popular open-source projects Hadoop, HBase, HDFS, Mesos, Spark, and MapReduce and apply five machine learning classifiers Multinomial Naive Bayes, Decision Tree, Logistic Regression, Random Forest, AdaBoost to automatically predict bug priority using the title, description, and summary of the bug report. We use tf-idf to extract useful features from the bug reports and employ precision, recall, and F1-score for measuring the performance of the classifiers. A stratified 10-fold cross-validation technique is used for model evaluation and the results are averaged over all 10 folds. We find that machine learning applied to bug priority prediction provides excellent results and can be used to significantly reduce the time involved in the bug prioritization process. From our experiments, we observe that no single classifier consistently performs best on all priority levels and metrics across all datasets. However, trends from results show that Multinomial Naive Bayes gives well balanced performance and is also fast to train and test. Logistic Regression and AdaBoost also performed well and are potential alternatives.**

Keywords—Bug Priority Prediction, Mining Bug Repository, Text Classification, Machine Learning, Software Maintenance

I. Introduction

Software systems have grown complicated over the years with each software project spanning millions of Lines of Code (LOC) at times [1-3]. For popular software systems, the number of daily submitted bug reports is high. As the size of projects increase, we also see an increase in reported bugs due in part to the increase in complexity and distributed, collaborative nature of programming. Companies grapple with handling all these bug reports as the available manpower is limited. One way to deal with this problem is to deal with the important bugs at first and then move on to the not so necessary bug fixes or optional improvements. However, assigning priority to these incoming bugs manually is a time-consuming task. It can be automated by implementing a bug priority system that assigns a priority to each bug so that it can be resolved accordingly.

The present way of assigning priorities to a bug report is to manually assign it by a developer who is very familiar with the project. This is time, and effort consuming and robs valuable time from the developer. Automating this process has been tried by much past work [4-10]. By automating the bug priority assigning process, we can also alleviate the individual bias of the developer. The typical priority levels are Blocker, Critical, Major, Minor, and Trivial, ordered in descending order of priorities. Previous work in this field has focused on Eclipse and Firefox datasets. In this paper, we demonstrate bug priority prediction on six new datasets Hadoop, HBase, HDFS, Mesos, Spark, and MapReduce [11-16], collected from the bug tracking system JIRA. We apply five different machine learning techniques, namely Multinomial Naive Bayes, Decision Tree, Logistic Regression, Random Forest, AdaBoost to predict bug priority and compare their performance.

II. Related Work

Bug priority prediction is usually treated as a classification problem with the labels being the priority of the bug. Various studies have tried automatically assigning priority to bugs and some have tried machine learning methods to the problem as well. A significant contribution to the problem of automated bug priority prediction is from Yuan Tian et al.'s 'Automated prediction of bug report priority using multi-factor analysis' [4] which introduced the DRONE framework. DRONE used different properties of bug reports like temporal, textual, author, related-report, severity, and product information in order to predict bug priority. Like most papers in the area of automated bug priority prediction, DRONE used the Eclipse dataset which is highly imbalanced. The performance measure used in DRONE is F-Measure which is an appropriate metric for judging models on imbalanced datasets. Qasim Umer et al. [17] build upon DRONE and improve its performance by assigning an emotion value to each bug report based on emotion words presented in the bug report description. Our work introduces 6 new datasets that enable us to judge the generalization ability of our techniques. We also use the same F-Measure along with precision and recall as metrics because the datasets used in our paper are also imbalanced. It also makes our results comparable with previous work in the field.

Authorized licensed use limited to: San Francisco State Univ. Downloaded on June 25,2021 at 22:18:25 UTC from IEEE Xplore. Restrictions apply.

Uddin et al. [33] conducted a survey on automated bug prioritization using articles from the year 2000 to 2015. Their results showed that researchers mostly treated it as a classification problem and used techniques like Naive Bayes, Support Vector Machine, k-Nearest Neighbors, Decision Trees, Random Forest, etc. for classification. The top three commonly used evaluation metrics were Precision, Recall, and F-Measure, and the Eclipse and Mozilla datasets were found to be heavily used in previous studies. In our study, we also formulated bug priority prediction as a classification problem and employed the popular metrics and classifiers as mentioned in their study.

Wei Zhang et al. [5] treated priority prediction as a regression task and argued that the priority levels are ordered and the ordinal knowledge will be lost in a classification setting. Their dataset consisted of a total of 82,299 bugs from a software project, covering reports from 2004 to 2017. They extracted features from bug reports using topic modeling and trained a Deep Neural Network (DNN) to predict the priority of a bug. The study used Mean Absolute Error (MAE) between predictions and ground truth priorities to measure performance as they explain that MAE may be more aligned with the human viewpoint. They also used Precision, Recall, and F-Measure metrics to be more comparable with the previous studies and showed that their approach was effective in bug priority prediction.

Another closely related problem is that of bug triaging, which deals with assigning bugs to the relevant developer. Prior work in this field used bug reports, bug priority, severity, developer experience, etc to try and automate bug triaging. Plenty of past studies have dealt with this topic [19-26]. Automating both the task of bug triaging and bug priority prediction is of much use to the software developer community.

III. Proposed Approach and Experimental Design

A. Independent and Dependent Variables

Since the purpose here is to predict or assign a priority to a bug report, priority is the dependent variable in our experiment, also called the target variable. To predict the target variable, we select a set of features that are the contents of the bug report. We take the title of the report, its description, and the summary of the report as our features set. These are called independent variables in our experiment. We do not consider any other information such as the author of the bug report, project information, version, time, comments, etc. in this study.

B. Dataset Collection

For the sake of reproducibility and ease of access, we selected 6 popular open-source projects and collected bug reports from the issue and project tracking software JIRA [11-16]. We automated the extraction and storage of reports from JIRA using Python. Summary statistics of the collected reports from all projects are given in Table 1.

TABLE I. Summary Statistics of Bug Reports

Project Name	# bug reports
Hadoop	17,268
HBase	25,051
HDFS	15,445
Mesos	10,160
Spark	32,735
MapReduce	7,302

C. Dataset Preprocessing

The extracted raw reports need to be processed before they can be used in further steps of feature extraction and modelling. The raw reports are stored in the XML data format. Firstly, we extract the title, description, summary, and priority from all the reports. All 4 fields are of type string. We skip all the reports where priority is not given. Null values in the description and summary are filled with empty strings. A new column "reports" is created by concatenating title, description, and summary. We clean the reports by removing HTML tags, hex code, links, special characters, converting all text to lowercase. Stop words and punctuations are removed from the reports as they appear frequently and do not provide any useful information. Tokenization is the step of breaking up the text document into a sequence of words. We perform it with the help of Stanford's NLTK package [27]. The datasets in original form were pretty imbalanced with one priority level - "Major" being assigned to more than half of the reports in the dataset. Imbalanced datasets introduce bias in the models trained which leads to poor performance. [31] Some of the ways to solve this include oversampling and undersampling. To implement undersampling, we deleted some reports randomly making the target distribution balanced. The same sequence of steps is performed for each dataset independently.

D. Feature Extraction

Machine learning cannot work with text data directly. We need to encode it into some feature vectors of numbers. Two of the popular approaches to extract features from text data are Bag of Words (BOW) and Term frequency-inverse document frequency (tf-idf) [28]. The common issue with BOW is that commonly occurring but less important words will have very high counts and some less occurring but more important, for example, FileNotFoundException, RuntimeException, etc. words that are highly correlated to priority, will have low counts. tf-idf is robust to this effect and focuses on words that are frequent in documents but not across all documents. It is a quite popular option and we also chose tf-idf for our experiments. Previous studies [17, 29, 30] also used tf-idf in their experiments. When building the vocabulary using tf-idf, we ignored the terms that had a document frequency lower than 5. We also used unigrams and bigrams while building vocabulary. We experimented with different values of document frequency threshold and n-grams and found these to be working well.

E. Performance Measures

To evaluate the effectiveness of different machine learning classifiers, we chose the commonly used classification metrics - precision, recall, and f-measure. Accuracy was not chosen because of its poor performance on class imbalanced problems. A simple classifier predicting the majority class at all times would score the same accuracy percentage as the percentage of the majority class in the dataset. We also used the average of these metrics, weighted by the number of true instances for each label, or also called support.

Authorized licensed use limited to: San Francisco State Univ. Downloaded on June 25,2021 at 22:18:25 UTC from IEEE Xplore. Restrictions apply.

F. Training and Evaluation

We chose 5 of the popular machine learning classifiers from the Scikit-learn[18] library, namely Multinomial Naive Bayes, Decision Tree Classifier, Logistic Regression, Random Forest Classifier (bagging technique), and AdaBoost Classifier (boosting technique) for our experiments as they perform well in text classification tasks [32]. To evaluate our classifiers, we selected a k-fold cross-validation technique. The value of k was taken as 10, making it a 10-fold cross-validation procedure where each time we split the dataset into 10 sets or folds and hold out 1 set as test data and the remaining 9 sets as the training data. A rigorous cross-validation strategy gives us the confidence that the models are not overfitted and generalize well over unseen data. We also used stratification in k-fold instead of random sampling to make the class distribution - all 5 priority labels, similar in train and test data. It helped us to reduce the issues caused due to imbalance and worked well for all classifiers. We averaged the scores (precision, recall, f-measure, and weighted average) over all 10-folds and used them for evaluation.

IV. Results and Discussions

We obtained the results of running 5 different machine learning classifiers on 6 bug reports datasets and compiled them in Tables 2-7. Each table corresponds to a single dataset and contains scores achieved by each classifier on that dataset. The different classifiers are listed in the first column of each table. The reported scores are - precision, recall, and f-measure, for each priority level and also a weighted average is provided. The columns after the classifiers column of the table represent priority levels and at last is the weighted average column where we report the average of the metrics (precision, recall, and f-measure), weighted by support. The results for each dataset are averaged over stratified 10-folds cross-validation scores for each classifier.

The results tables may be read in the following manner - we select the priority level or weighted average, fix the evaluation metric (Precision, Recall, or F1-score), and then compare the values obtained by different classifiers. For instance, in Table 2, we can select the Major priority level and F1-score metric to compare and see that Multinomial Naive Bayes performs best among all classifiers. Similarly, for the weighted average and F1-score metric in Table 2, we notice that Logistic Regression outperforms other classifiers.

TABLE II. 10-fold Cross-validation Scores on Hadoop

		Blocker	Critical	Major	Minor	Trivial	Average
Logistic Regression	**Precision**	0	0	0.631	0.519	0.05	0.444
	Recall	0	0	0.731	0.69	0.002	0.545
	F1-score	0	0	0.653	0.568	0.003	0.47
Random Forest	**Precision**	0	0	0.625	0.503	0	0.432
	Recall	0	0	0.681	0.71	0	0.532
	F1-score	0	0	0.626	0.567	0	0.457
Decision Tree	**Precision**	0.131	0	0.596	0.446	0.292	0.432
	Recall	0.035	0	0.434	0.77	0.035	0.913
	F1-score	0.054	0	0.486	0.558	0.059	0.41
Multinomial NB	**Precision**	0.272	0	0.63	0.515	0	0.468
	Recall	0.013	0	0.742	0.676	0	0.54
	F1-score	0.023	0	0.654	0.553	0	0.46
AdaBoost	**Precision**	0.323	0.158	0.581	0.432	0.37	0.457
	Recall	0.045	0.028	0.454	0.723	0.107	0.459
	F1-score	0.077	0.047	0.49	0.535	0.161	0.411

TABLE III. 10-fold Cross-validation Scores on HBase

		Blocker	Critical	Major	Minor	Trivial	Average
Logistic Regression	**Precision**	0	0.438	0.517	0.481	0.3	0.442
	Recall	0	0.023	0.674	0.656	0.003	0.484
	F1-score	0	0.04	0.571	0.543	0.006	0.413
Random Forest	**Precision**	0	0.325	0.483	0.439	0	0.375
	Recall	0	0.014	0.511	0.721	0	0.451
	F1-score	0	0.028	0.491	0.544	0	0.381
Decision Tree	**Precision**	0.131	0	0.596	0.446	0.292	0.432

Authorized licensed use limited to: San Francisco State Univ. Downloaded on June 25,2021 at 22:18:25 UTC from IEEE Xplore. Restrictions apply.

	Recall	0.035	0	0.434	0.77	0.035	0.463
	F1-score	0.054	0	0.486	0.558	0.059	0.408
Multinomial NB	**Precision**	0.2	0.378	0.528	0.475	0.025	0.426
	Recall	0.002	0.038	0.654	0.663	0.001	0.485
	F1-score	0.004	0.065	0.571	0.543	0.002	0.413
AdaBoost	**Precision**	0.364	0.304	0.43	0.423	0.337	0.4
	Recall	0.094	0.051	0.422	0.674	0.081	0.418
	F1-score	0.144	0.088	0.426	0.518	0.131	0.38

TABLE IV. 10-FOLD CROSS-VALIDATION SCORES ON HDFS

		Blocker	**Critical**	**Major**	**Minor**	**Trivial**	**Average**
Logistic Regression	**Precision**	0	0	0.549	0.525	0.317	0.441
	Recall	0	0	0.681	0.665	0.01	0.527
	F1-score	0	0	0.599	0.577	0.02	0.46
Random Forest	**Precision**	0	0	0.501	0.513	0	0.398
	Recall	0	0	0.752	0.525	0	0.5
	F1-score	0	0	0.598	0.511	0	0.43
Decision Tree	**Precision**	0.114	0.19	0.445	0.47	0.394	0.407
	Recall	0.028	0.034	0.813	0.284	0.076	0.442
	F1-score	0.044	0.082	0.575	0.348	0.126	0.38
Multinomial NB	**Precision**	0	0.3	0.555	0.523	0.05	0.45
	Recall	0	0.006	0.697	0.655	0.002	0.529
	F1-score	0	0.009	0.609	0.572	0.004	0.46
AdaBoost	**Precision**	0.205	0.276	0.465	0.479	0.356	0.424
	Recall	0.046	0.064	0.694	0.421	0.16	0.458
	F1-score	0.076	0.097	0.557	0.445	0.214	0.42

TABLE V. 10-FOLD CROSS-VALIDATION SCORES ON MESOS

		Blocker	**Critical**	**Major**	**Minor**	**Trivial**	**Average**
Logistic Regression	**Precision**	0.2	0.2	0.546	0.484	0	0.425
	Recall	0.007	0.006	0.68	0.677	0	0.505
	F1-score	0.013	0.01	0.596	0.557	0	0.43
Random Forest	**Precision**	0.213	0.153	0.594	0.443	0	0.424
	Recall	0.018	0.011	0.52	0.782	0	0.484
	F1-score	0.032	0.019	0.541	0.565	0	0.42
Decision Tree	**Precision**	0.259	0.265	0.567	0.419	0.089	0.428
	Recall	0.123	0.138	0.339	0.769	0.017	0.435
	F1-score	0.163	0.176	0.435	0.542	0.027	0.39
Multinomial NB	**Precision**	0.267	0.25	0.618	0.453	0	0.454
	Recall	0.015	0.011	0.495	0.851	0	0.498
	F1-score	0.029	0.02	0.54	0.59	0	0.42
AdaBoost	**Precision**	0.322	0.261	0.507	0.422	0.185	0.416
	Recall	0.077	0.195	0.551	0.543	0.054	0.436
	F1-score	0.122	0.215	0.506	0.456	0.08	0.4

Authorized licensed use limited to: San Francisco State Univ. Downloaded on June 25,2021 at 22:18:25 UTC from IEEE Xplore. Restrictions apply.

TABLE VI. 10-FOLD CROSS-VALIDATION SCORES ON SPARK

		Blocker	Critical	Major	Minor	Trivial	Average
Logistic Regression	**Precision**	0.1	0	0.438	0.417	0	0.328
	Recall	0.001	0	0.577	0.558	0	0.421
	F1-score	0.002	0	0.49	0.465	0	0.354
Random Forest	**Precision**	0.45	0	0.421	0.421	0	0.321
	Recall	0.01	0	0.735	0.385	0	0.383
	F1-score	0.019	0	0.528	0.387	0	0.313
Decision Tree	**Precision**	0.142	0.156	0.385	0.402	0.243	0.332
	Recall	0.024	0.022	0.82	0.246	0.008	0.382
	F1-score	0.041	0.037	0.523	0.294	0.016	0.299
Multinomial NB	**Precision**	0.401	0.238	0.445	0.42	0	0.375
	Recall	0.012	0.006	0.577	0.565	0	0.426
	F1-score	0.024	0.01	0.489	0.468	0	0.359
AdaBoost	**Precision**	0.091	0.125	0.4	0.419	0.125	0.335
	Recall	0.018	0.009	0.821	0.239	0.015	0.398
	F1-score	0.027	0.015	0.535	0.293	0.026	0.314

TABLE VII. 10-FOLD CROSS-VALIDATION SCORES ON MAPREDUCE

		Blocker	Critical	Major	Minor	Trivial	Average
Logistic Regression	**Precision**	0.422	0	0.519	0.408	0	0.39
	Recall	0.036	0	0.859	0.448	0	0.481
	F1-score	0.063	0	0.647	0.418	0	0.392
Random Forest	**Precision**	0.223	0.312	0.481	0.386	0	0.379
	Recall	0.038	0.053	0.967	0.236	0	0.46
	F1-score	0.062	0.093	0.641	0.286	0	0.355
Decision Tree	**Precision**	0.248	0.233	0.487	0.374	0.224	0.379
	Recall	0.16	0.09	0.853	0.219	0.046	0.434
	F1-score	0.188	0.121	0.621	0.275	0.073	0.368
Multinomial NB	**Precision**	0.397	0.4213	0.526	0.419	0	0.437
	Recall	0.088	0.042	0.863	0.429	0	0.486
	F1-score	0.188	0.121	0.621	0.275	0	0.368
AdaBoost	**Precision**	0.297	0.326	0.509	0.345	0.286	0.397
	Recall	0.135	0.15	0.769	0.299	0.116	0.433
	F1-score	0.174	0.191	0.609	0.317	0.164	0.389

It can be observed from results that Logistic Regression and Multinomial Naive Bayes consistently outperforms other classifiers on weighted average of Precision, Recall, and F1-score. However, Logistic Regression does not perform well on "Blocker" and "Critical" priority levels which are generally considered as high priority. AdaBoost on the other hand performs better than other classifiers on "Blocker" and "Critical" levels. The classifiers that also perform well on these two priority levels are Multinomial Naive Bayes and Decision Trees. Random Forest performs badly for "Blocker", "Critical", and "Trivial" levels, as evident from precision, recall, and F1-score.

Among all classifiers, the Multinomial Naive Bayes classifier was the fastest to train and test on all datasets and it also achieves good metric scores. We find that the Multinomial Naive Bayes and Logistic Regression classifier performs best on weighted average scores. AdaBoost and Multinomial Naive Bayes give well balanced performance on all classes and are not just focused on some specific priority level. There's no single classifier that consistently performs best on all priority levels and metrics across all the datasets. However, from the general trends observed from experiment results, it is clear that Multinomial Naive Bayes is a good choice for bug prioritization as it gives well

Authorized licensed use limited to: San Francisco State Univ. Downloaded on June 25,2021 at 22:18:25 UTC from IEEE Xplore. Restrictions apply.

adjusted performance on all priority levels and metrics. It is also fast and easy to train.

V. Conclusion

In order to prioritize bugs, we need the bug priority assignee to be very familiar with the codebase and they end up spending a lot of time assigning priority. To relieve them of this time-consuming work and to avoid individual bias creeping into the process, we propose to automate the bug priority assignment using machine learning methods on bug reports of six open-source datasets. We collected the bug reports of these projects from bug tracking software and preprocessed them for use. Some useful features were extracted from the bug reports and five selected machine learning classifiers were chosen for modelling. Our validation strategy was a stratified 10-fold cross-validation technique and we averaged precision, recall, F1-score, and weighted averaged (weighted by support) over all 10 folds. After finding the experiment results, we conclude that no single classifier is best on all priority levels and metrics across all the datasets and each classifier performs differently. Some of them get better weighted scores while others perform well on individual priority levels. However, Multinomial Naive Bayes was found to be performing well on all priority levels and metrics across all datasets and it was also fast to train and test.

References

[1] E. Ng'ang'a and I. Tonui, "A Survey on Software Sizing for Project Estimation," International Journal of Software Engineering and Knowledge Engineering, vol. 5, p. 56, Jan. 2015.

[2] T. S. S. Angel, N. Snehalatha, S. Amutha, and J. T. M. Dhas, "IMPORTANCE OF SOFTWARE SIZING IN SOFTWARE PROJECT MANAGEMENT: A STUDY," p. 6.

[3] Z. Jiang, P. Naudé, and B. Jiang, "The Effects of Software Size on Development Effort and Software Quality," vol. 1, no. 4, p. 5.

[4] "Automated prediction of bug report priority using multi-factor analysis | SpringerLink." https://link.springer.com/article/10.1007/s10664-014-9331-y (accessed Nov. 09, 2020).

[5] W. Zhang and C. Challis, "Automatic Bug Priority Prediction Using DNN Based Regression," in Advances in Natural Computation, Fuzzy Systems and Knowledge Discovery, Cham, 2020, pp. 333–340, doi: 10.1007/978-3-030-32456-8_36.

[6] A. Lamkanfi, S. Demeyer, Q. D. Soetens, and T. Verdonck, "Comparing Mining Algorithms for Predicting the Severity of a Reported Bug," in 2011 15th European Conference on Software Maintenance and Reengineering, Mar. 2011, pp. 249–258, doi: 10.1109/CSMR.2011.31.

[7] Q. Umer, H. Liu, and Y. Sultan, "Emotion Based Automated Priority Prediction for Bug Reports," IEEE Access, vol. 6, pp. 35743–35752, 2018, doi: 10.1109/ACCESS.2018.2850910.

[8] "Information Retrieval Based Nearest Neighbor Classification for Fine-Grained Bug Severity Prediction - IEEE Conference Publication." https://ieeexplore.ieee.org/document/6385117 (accessed Nov. 09, 2020).

[9] P. A. Choudhary and D. S. Singh, "Neural Network Based Bug Priority Prediction Model using Text Classification Techniques," International Journal of Advanced Research in Computer Science, p. 5, 2017.

[10] A. Lamkanfi, S. Demeyer, E. Giger, and B. Goethals, "Predicting the severity of a reported bug," in 2010 7th IEEE Working Conference on Mining Software Repositories (MSR 2010), May 2010, pp. 1–10, doi: 10.1109/MSR.2010.5463284.

[11] Spark JIRA bug tracker https://issues.apache.org/jira/projects/SPARK/issues (Accessed Nov. 09, 2020).

[12] HBase JIRA bug tracker https://issues.apache.org/jira/projects/HBASE/issues (Accessed Nov. 09, 2020).

[13] HDFS JIRA bug tracker https://issues.apache.org/jira/projects/HDFS/issues (Accessed Nov. 09, 2020).

[14] MapReduce JIRA bug tracker https://issues.apache.org/jira/projects/MAPREDUCE/issues (Accessed Nov. 09, 2020).

[15] Mesos JIRA bug tracker https://issues.apache.org/jira/projects/MESOS/issues (Accessed Nov. 09, 2020).

[16] Hadoop JIRA bug tracker https://issues.apache.org/jira/projects/HADOOP/issues (Accessed Nov. 09, 2020).

[17] Umer, H. Liu, and Y. Sultan, "Emotion Based Automated Priority Prediction for Bug Reports," IEEE Access, vol. 6, pp. 35743–35752, 2018, doi: 10.1109/ACCESS.2018.2850910.

[18] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," MACHINE LEARNING IN PYTHON, p. 6.

[19] T. S. Gadge and N. Mangrulkar, "Approaches for automated bug triaging: A review," in 2017 International Conference on Innovative Mechanisms for Industry Applications (ICIMIA), Bengaluru, India, Feb. 2017, pp. 158–161, doi: 10.1109/ICIMIA.2017.7975592.

[20] P. Bhattacharya, I. Neamtiu, and C. R. Shelton, "Automated, highly-accurate, bug assignment using machine learning and tossing graphs," Journal of Systems and Software, vol. 85, no. 10, pp. 2275–2292, Oct. 2012, doi: 10.1016/j.jss.2012.04.053.

[21] J. Park, M.-W. Lee, J. Kim, S. Hwang, and S. Kim, "COSTRIAGE: A Cost-Aware Triage Algorithm for Bug Reporting Systems," p. 6.

[22] "Effective Bug Triage – A Framework," Procedia Computer Science, vol. 48, pp. 114–120, Jan. 2015, doi: 10.1016/j.procs.2015.04.159.

[23] M. Alenezi, K. Magel, and S. Banitaan, "Efficient Bug Triaging Using Text Mining," JSW, vol. 8, no. 9, pp. 2185–2190, Sep. 2013, doi: 10.4304/jsw.8.9.2185-2190.

[24] A. Tamrawi, T. T. Nguyen, J. Al-Kofahi, and T. N. Nguyen, "Fuzzy set-based automatic bug triaging: NIER track," in 2011 33rd International Conference on Software Engineering (ICSE), May 2011, pp. 884–887, doi: 10.1145/1985793.1985934.

[25] T. Zhang, G. Yang, B. Lee, and I. Shin, "Role Analysis-based Automatic Bug Triage Algorithm," Information Processing Society of Japan, p. 4.

[26] W. Zou, Y. Hu, J. Xuan, and H. Jiang, "Towards Training Set Reduction for Bug Triage," in 2011 IEEE 35th Annual Computer Software and Applications Conference, Jul. 2011, pp. 576–581, doi: 10.1109/COMPSAC.2011.80.

[27] E. Loper and S. Bird, "NLTK: the Natural Language Toolkit," in Proceedings of the ACL-02 Workshop on Effective tools and methodologies for teaching natural language processing and computational linguistics - Volume 1, USA, Jul. 2002, pp. 63–70, doi: 10.3115/1118108.1118117.

[28] R. Stohler, Document Embedding Models: A Comparison with Bag-of-Words. Zurich: University of Zurich, 2018.

[29] J. Xuan, H. Jiang, Z. Ren, and W. Zou, "Developer prioritization in bug repositories," in 2012 34th International Conference on Software Engineering (ICSE), Jun. 2012, pp. 25–35, doi: 10.1109/ICSE.2012.6227209.

[30] P. Bhattacharya and I. Neamtiu, "Fine-grained incremental learning and multi-feature tossing graphs to improve bug triaging," in 2010 IEEE International Conference on Software Maintenance, Sep. 2010, pp. 1–10, doi: 10.1109/ICSM.2010.5609736.

[31] N. Japkowicz and S. Stephen, "The class imbalance problem: A systematic study," Intell. Data Anal., vol. 6, no. 5, pp. 429–449, Oct. 2002.

[32] K. Kowsari, K. J. Meimandi, M. Heidarysafa, S. Mendu, L. E. Barnes, and D. E. Brown, "Text Classification Algorithms: A Survey," Information, vol. 10, no. 4, p. 150, Apr. 2019, doi: 10.3390/info10040150.

[33] J. Uddin, R. Ghazali, M. M. Deris, R. Naseem, and H. Shah, "A survey on bug prioritization," Artif. Intell. Rev., vol. 47, no. 2, pp. 145–180, Feb. 2017, doi: 10.1007/s10462-016-9478-6.

Authorized licensed use limited to: San Francisco State Univ. Downloaded on June 25,2021 at 22:18:25 UTC from IEEE Xplore. Restrictions apply.